

SETUP GUIDE

Getting mnsh1 up on Oracle Cloud

A free 2-core / 12 GB ARM server, grabbed automatically by GitHub Actions. This guide walks you through the GitHub secrets, the Oracle terminal and the login username, step by step.

Oracle's Always Free tier includes an Ampere ARM virtual machine, but in busy regions the console just says *Out of capacity* every time you click Create. The repo you've been given fixes that: a GitHub Actions job retries the launch every five minutes, and the moment a slot opens it creates the instance for you. Your part is a one-time setup of roughly 15 minutes. After that, you wait.

What you'll end up with

Name	mnsh1
Shape	VM.Standard.A1.Flex (Ampere ARM, aarch64)
CPU	2 OCPU
Memory	12 GB
Storage	200 GB boot volume
OS	Ubuntu 24.04 Minimal, ARM build
Login	user ubuntu , SSH key only: <code>ssh -i ~/.ssh/oci ubuntu@<ip></code>
Cost	\$0. Stays inside the Always Free limits (4 OCPU / 24 GB / 200 GB storage).

Before you start, you need

- An **Oracle Cloud account** (cloud.oracle.com). Sign-up asks for a card for identity verification; nothing in this guide is billed.
- A **GitHub account**.
- A **terminal** on your computer: Terminal on macOS, any shell on Linux, or PowerShell on Windows 10/11. All of them ship with `ssh` and `ssh-keygen`.
- About 15 minutes, and a scratch text file to paste values into as you go (outside the repo folder).

The plan

1. **Collect your Oracle identity:** username, user OCID, tenancy OCID, region.
2. **Create an API key.** This is how GitHub logs in to Oracle as you.
3. **Open the Oracle terminal** (Cloud Shell) and look up your subnet.
4. **Make an SSH key** on your computer.
5. **Put the repo on GitHub** as a private repo.
6. **Add the seven GitHub secrets.**
7. **Run the workflow once** and read the log.
8. **Wait.** When mnsh1 lands, log in as **ubuntu**.

1. Your Oracle identity: username and OCIDs

Everything in Oracle Cloud is identified by an **OCID**, a long string starting with `ocid1..` You need two of them (your user and your tenancy), plus your region. Oracle calls these things by slightly different names in different places, so here is exactly where each one lives.

1. Sign in at **cloud.oracle.com**. The first screen asks for your **Cloud Account Name** (the tenancy name you chose at sign-up), then your **username** and password. Your username is normally the email you signed up with.
2. **Region**. The top bar shows the current region, e.g. *India West (Mumbai)*. Free-tier VMs can only be created in your **home region**, so leave it as it is. The identifier you'll need looks like `ap-mumbai-1`; it appears in the API-key preview in section 2, so there is no need to hunt for it now.
3. **User OCID**. Click the profile icon (top right) **My profile**. The page shows your username and, under it, **OCID** with a *Copy* link. Paste it next to `OCI_CLI_USER` in your scratch file.
4. **Tenancy OCID**. Profile icon **Tenancy: <your name>**. Copy its **OCID** next to `OCI_CLI_TENANCY`.

What the console calls it	What it is	Secret name
Cloud Account Name / Tenancy name	The account itself. Used only to sign in.	not a secret
Username	Who you are, usually your email. Used only to sign in.	not a secret
User OCID	Machine ID of your user	<code>OCI_CLI_USER</code>
Tenancy OCID	Machine ID of the account	<code>OCI_CLI_TENANCY</code>
Region identifier	Where the VM lives, e.g. <code>ap-mumbai-1</code>	<code>OCI_CLI_REGION</code>

Tip: OCIDs are identifiers, not passwords. The only true secret in this whole setup is the private key file from section 2.

2. Create an API key

The GitHub job talks to Oracle through the OCI API. Instead of your password it uses an **API signing key**: a private key that lives in a GitHub secret, and a matching public key registered on your Oracle user.

1. Profile icon **My profile**. Find **API keys**. In the newer console layout it sits under the **Tokens and keys** tab; in the older one it is in the **Resources** list on the left.
2. Click **Add API key** and choose **Generate API key pair**.
3. Click **Download private key**. A file ending in `.pem` lands in your Downloads. Do **not** set a passphrase. (You may also download the public key; you won't need it.)
4. Click **Add**. A **Configuration file preview** appears. It looks like this:

```
[DEFAULT]
user=ocid1.user.oc1..aaaaaaaa...
fingerprint=3f:1a:9c:...:e2
tenancy=ocid1.tenancy.oc1..aaaaaaaa...
region=ap-mumbai-1
key_file=<path to your private keyfile> # TODO
```

- Copy the four values `user`, `fingerprint`, `tenancy` and `region` into your scratch file. They become `OCI_CLI_USER`, `OCI_CLI_FINGERPRINT`, `OCI_CLI_TENANCY` and `OCI_CLI_REGION`.
- Open the downloaded `.pem` in a **plain-text** editor (TextEdit in plain-text mode, Notepad, VS Code). The whole file, from `-----BEGIN PRIVATE KEY-----` to `-----END PRIVATE KEY-----` inclusive, becomes `OCI_CLI_KEY_CONTENT`.

Careful: Never copy the `.pem` into the repo folder, and never commit it. Anyone holding that file plus your OCIDs controls your entire Oracle account. It goes into a GitHub secret and nowhere else. Once `mnsh1` is up and the workflow is disabled, you can delete the API key under **My profile > API keys** to revoke it.

Tip: A user can have up to three API keys. If you lose the `.pem`, delete that key, generate a new one, and update the fingerprint and key-content secrets.

3. The Oracle terminal (Cloud Shell)

Oracle's console has a built-in terminal called **Cloud Shell**. It is a small Linux box in your browser with the `oci` command-line tool pre-installed and **already logged in as you**, so there is nothing to configure. It is the quickest way to look up OCIDs without clicking around, and to sanity-check things later.

- In the console, click the **Developer tools** icon in the top bar (the `>_` terminal symbol, near the search bar and region name) **Cloud Shell**. The first launch takes 30–60 seconds.
- Cloud Shell already knows who you are. Print it and compare with sections 1–2:

```
echo "user:    $OCI_CS_USER_OCID"
echo "tenancy: $OCI_TENANCY"
echo "region:  $OCI_REGION"
```

If a variable prints empty, just use the values you copied from the console.

- Confirm the CLI works and see your availability domain(s). Mumbai shows a single one; that is expected.

```
oci iam availability-domain list --compartment-id "$OCI_TENANCY" \
  --query 'data[].name' --output table
```

Find (or create) your network and copy the subnet OCID

The instance has to be placed in a subnet of a **Virtual Cloud Network** (VCN). New accounts usually have none.

- Console menu (the three lines, top left) **Networking > Virtual cloud networks**. If the list is empty: **Start VCN Wizard > Create VCN with Internet Connectivity > Start VCN Wizard**, give it a name such as `mnsh-vcn`, keep the defaults, **Next > Create**. It takes about a minute.
- Back in Cloud Shell, list the subnets and pick the **public** one:

```
oci network subnet list --compartment-id "$OCI_TENANCY" --output table \
  --query 'data[].[name:"display-name", noPublicIp:"prohibit-public-ip-on-vnic", id:id]'
```

The row whose `noPublicIp` column says `false` is the public subnet. Its `id` (`ocid1.subnet.oc1...`) is `OCI_SUBNET_ID`. Through the console instead: **open the VCN > the subnet named public subnet-... > OCID > Copy**.

- Optional: see the Ubuntu ARM images the workflow chooses from. You don't need to set anything; it picks the newest one whose name contains *Minimal*. Only if you want to pin one exact image do you put its id in the optional OCI_IMAGE_ID secret.

```
oci compute image list --compartment-id "$OCI_TENANCY" \
  --operating-system "Canonical Ubuntu" --operating-system-version "24.04" \
  --shape VM.Standard.A1.Flex --sort-by TIMECREATED --sort-order DESC \
  --query 'data[].{name:"display-name", id:id}' --output table
```

Tip: Cloud Shell's home directory persists between sessions, but the session itself times out after about 20 idle minutes. Just reopen it.

Optional: the OCI CLI on your own computer

Not required for this guide, but handy if you'd rather not use the browser terminal:

```
# macOS
brew install oci-cli
# Linux / Windows (needs Python 3)
pip install oci-cli

oci setup config # asks for user OCID, tenancy OCID, region; can generate the key
```

oci setup config writes ~/.oci/config with the same user / fingerprint / tenancy / region values the GitHub secrets use, plus ~/.oci/oci_api_key.pem. If you let it generate a new key, upload the matching oci_api_key_public.pem under **My profile > API keys > Add API key > Paste a public key**, otherwise Oracle will not accept that key.

4. An SSH key for logging in

This is how **you** will log in to mnsh1. The public half goes to Oracle (through a GitHub secret) and is baked into the server when it is created; the private half never leaves your computer.

```
# macOS / Linux (Terminal) -- Windows 10/11 (PowerShell) is the same command
ssh-keygen -t ed25519 -f ~/.ssh/oci -C "mnsh1"
# press Enter twice for no passphrase (or set one; you'll type it at every login)

cat ~/.ssh/oci.pub # Windows: Get-Content $HOME\.ssh\oci.pub
```

The output is a single line starting with ssh-ed25519 AAAA and ending with mnsh1. That whole line is SSH_PUBLIC_KEY.

Careful: Two different keys, two different jobs. The Oracle API key (.pem) authenticates **GitHub to Oracle's API**. The SSH key (~/.ssh/oci) authenticates **you to the server**. OCI_CLI_KEY_CONTENT gets the .pem; SSH_PUBLIC_KEY gets the .pub line.

Tip: Want a second person to be able to log in too? Put their public key on a second line of the SSH_PUBLIC_KEY secret. One key per line.

Windows: if PowerShell says ssh-keygen is not recognised, add it via **Settings > Apps > Optional features > Add a feature > OpenSSH Client**.

5. Put the repo on GitHub (private)

Scheduled workflows only run from the default branch of a repo **you** own on GitHub, so you need your own copy. Make it **private**: the secrets themselves are never exposed either way, but a private repo keeps the run history, the success issue and any slip-ups out of public view.

1. On github.com click + (top right) **New repository**. Name: oraclefck (anything works). Visibility: **Private**. Do **not** tick *Add a README*. Click **Create repository**.
2. Get the files onto your computer and push them to your new repo:

```
git clone https://github.com/manish-ach/plzStopSpamming.git oraclefck
cd oraclefck
git remote set-url origin https://github.com/<your-github-username>/oraclefck.git
git push -u origin main
```

or, with the GitHub CLI, create-and-push in one go:
gh repo create oraclefck --private --source . --push

3. Check on GitHub that you can see four things: README.md, launch.sh, .gitignore and .github/workflows/retry.yml. If the .github folder is missing, the workflow doesn't exist. Use git rather than the web drag-and-drop uploader, which skips hidden dot-folders.

Tip: Secrets are per-repository. Even if the person who sent you this added you as a collaborator on their repo, use your own copy with your own Oracle values.

6. Add the GitHub secrets

In your repo: **Settings > Secrets and variables > Actions** (the *Settings* tab is on the repo's top bar; *Secrets and variables* sits under *Security* in the left sidebar). Under **Repository secrets** click **New repository secret**, type the **Name** exactly as in the table, paste the **Secret**, click **Add secret**. Repeat for each row.

Secret name	What to paste	Where it came from
OCI_CLI_USER	User OCID, ocid1.user.oc1....	Section 1, or user= in the API-key preview
OCI_CLI_TENANCY	Tenancy OCID, ocid1.tenancy.oc1....	Section 1, or tenancy= in the preview
OCI_CLI_FINGERPRINT	16 pairs of hex separated by colons, e.g. 3f:1a:9c:...:e2	fingerprint= in the preview
OCI_CLI_KEY_CONTENT	The entire .pem file, BEGIN and END lines included, line breaks intact	Section 2, step 6
OCI_CLI_REGION	Region identifier, e.g. ap-mumbai-1	region= in the preview
OCI_SUBNET_ID	Public subnet OCID, ocid1.subnet.oc1....	Section 3
SSH_PUBLIC_KEY	The ssh-ed25519 AAAA... line	Section 4

Optional secrets

Secret name	When you'd set it
DISCORD_WEBHOOK	To get the public IP as a Discord message the moment mnsh1 lands. See below.
OCI_COMPARTMENT_ID	Only if you created a separate compartment and want the VM (and the subnet) there. Default is the root compartment, which is fine.
OCI_IMAGE_ID	Only to pin one exact image OCID (section 3). Normally leave it unset.

Common paste mistakes

- No quotes around any value.
- Secret names are case-sensitive: `OCI_CLI_USER`, not `oci_cli_user`.
- `OCI_CLI_REGION` is the identifier (`ap-mumbai-1`), not the display name (*India West (Mumbai)*).
- The key content must be multi-line, exactly as in the file. Select-all in the editor, copy, paste into the secret box. GitHub keeps the line breaks.
- GitHub never shows a secret again after saving. If in doubt, open it, click **Update**, and paste again.

Optional: Discord ping

In Discord: **Server Settings** > **Integrations** > **Webhooks** > **New Webhook**, pick the channel, **Copy Webhook URL**, and save it as the `DISCORD_WEBHOOK` secret. When mnsh1 lands you get a message with the IP and the instance OCID. Without it you still get the GitHub issue and email, and the IP is in the Oracle console.

Variables: already set for mnsh1

The size and name are baked into the workflow defaults: name **mnsh1**, **2 OCPU**, **12 GB**, **200 GB** boot volume, Ubuntu 24.04 Minimal ARM. You do not need to create any Variables. If you ever want different values, use the **Variables** tab next to Secrets: `OCI_DISPLAY_NAME`, `OCI_OCPUS`, `OCI_MEMORY_GB`, `OCI_BOOT_VOLUME_GB`, `OCI_OS_VERSION`.

7. Run it once and read the log

1. Repo **Actions** tab. If GitHub asks, click *I understand my workflows, go ahead and enable them*.
2. Left sidebar: **Grab OCI A1 free instance Run workflow** (right-hand side) **Run workflow** (green button).
3. After about ten seconds a run appears. Click it **launch**, then expand **Attempt launch**.

What the log says	What it means	What to do
Out of capacity in ... will retry	Auth and config are correct; Oracle simply has no free ARM slot right now. This is the normal state, sometimes for days.	Nothing. The cron keeps trying every 5 minutes.
Rate-limited by OCI (429)	Oracle asked us to slow down.	Nothing.
SUCCESS -- A1 instance is RUNNING	You got it.	Go to section 8.
Non-capacity error (red)	Something in the secrets or values is wrong. The real message is a few lines above it.	See the table below.

Fixing a red run

Error text contains	Likely cause	Fix
NotAuthenticated / 401	User OCID, fingerprint or key content don't match each other, or the key was added to a different user.	Simplest: delete the API key, generate a new one, re-paste OCI_CLI_USER, OCI_CLI_FINGERPRINT, OCI_CLI_KEY_CONTENT and OCI_CLI_TENANCY.
set OCI_SUBNET_ID or set SSH_PUBLIC_KEY	That secret is missing or its name is misspelled.	Add it with the exact name.
NotAuthorizedOrNotFound mentioning the subnet	Wrong subnet OCID, or the subnet lives in another compartment or region.	Copy the OCID again (section 3).
LimitExceeded	Your tenancy has no A1 quota, or another A1 instance already uses it.	Console menu Governance & Administration > Limits, Quotas and Usage : search <i>A1</i> and check that <i>Cores for Standard.A1 based VM and BM Instances</i> is 4. Delete other A1 VMs you don't need.
Could not resolve an image OCID	No Ubuntu 24.04 ARM image found by that name in your region.	Set OCI_IMAGE_ID to an id from the image command in section 3.
Invalid private key / could not deserialize	The .pem was pasted incompletely, or it has a passphrase.	Re-paste the whole file; regenerate without a passphrase if needed.

Tip: After the first successful manual run (green, even if it only said *Out of capacity*), the 5-minute schedule takes over on its own. GitHub cron runs are often 5–20 minutes late during busy hours; that is fine.

Careful: GitHub disables scheduled workflows after 60 days without a commit to the repo. If it is still hunting after two months, push a tiny commit (edit the README) or re-enable it from the Actions tab.

8. When mnsh1 lands

- GitHub opens an **issue** in your repo titled *...instance created* and emails you. The issue is deliberately generic and contains no IP.
- If you set DISCORD_WEBHOOK, the channel receives the **public IP** and the instance OCID.
- Otherwise the IP is in the console: menu **Compute > Instances > mnsh1**, field **Public IP address**. Or from Cloud Shell:

```
iid=$(oci compute instance list --compartment-id "$OCI_TENANCY" \
  --display-name mnsh1 --lifecycle-state RUNNING \
  --query 'data[0].id' --raw-output)
oci compute instance list-vnics --instance-id "$iid" \
  --query 'data[0].public-ip' --raw-output
```

- **Stop the retry job:** **Actions > Grab OCI A1 free instance**, the ... menu top right **Disable workflow**.
(The script will never create a second mnsh1 even if you forget, but there is no reason to keep it running.)

9. First login: the terminal and the username

The Ubuntu image comes with one ready-made user, **ubuntu**, and it can only log in with the SSH key you registered. There is no password and root login is disabled; use `sudo` for admin commands. (Oracle Linux images use **opc** instead. For Ubuntu images it is always **ubuntu**.)

```
ssh -i ~/.ssh/oci ubuntu@<PUBLIC_IP>
# first time only: type "yes" to accept the host fingerprint
```

If you get `Permission denied (publickey)`: you are pointing at the wrong private key file, using the wrong username (it is `ubuntu`, not `root`), or the `SSH_PUBLIC_KEY` secret had a typo. The last one means the key baked into the server is wrong; fix the secret, terminate the instance from the console and let the workflow create it again.

Save the connection as a shortcut

Add this to `~/.ssh/config` on your computer (create the file if it does not exist), and from then on `ssh mnsh1` is all you type:

```
Host mnsh1
  HostName <PUBLIC_IP>
  User ubuntu
  IdentityFile ~/.ssh/oci
```

First five minutes on the box

```
sudo apt update && sudo apt full-upgrade -y      # patch everything
sudo hostnamectl set-hostname mnsh1              # nicer prompt
sudo timedatectl set-timezone <Region/City>       # list: timedatectl list-timezones
sudo reboot                                       # log back in after ~30 s
```

Optional: your own username instead of ubuntu

Run these as **ubuntu**. The new user gets the same SSH key. Unlike **ubuntu**, it will be asked for the password you set here whenever it uses `sudo`.

```
sudo adduser --gecos "" mnsh                      # replace mnsh with the name you want
sudo usermod -aG sudo mnsh
sudo mkdir -p /home/mnsh/.ssh
sudo cp ~/.ssh/authorized_keys /home/mnsh/.ssh/
sudo chown -R mnsh:mnsh /home/mnsh/.ssh && sudo chmod 700 /home/mnsh/.ssh
# from your computer: ssh -i ~/.ssh/oci mnsh@<PUBLIC_IP>
```

Optional: opening a port (for example a web server)

Two firewalls sit in front of `mnsh1` and both default to *SSH only*. You have to open a port in both.

1. **Oracle side.** Menu **Networking > Virtual cloud networks > your VCN > Security Lists > Default Security List for ... > Add Ingress Rules**: Source CIDR `0.0.0.0/0`, IP Protocol **TCP**, Destination port range **80**. Repeat for **443** if you need HTTPS.
2. **Ubuntu side.** Oracle's Ubuntu images ship with iptables rules that reject everything except port 22:

```
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 80 -j ACCEPT
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 443 -j ACCEPT
sudo apt install -y iptables-persistent      # answer Yes to save the current rules
sudo netfilter-persistent save
```

10. Keeping it safe and free

- **The repo stays private.** Its files contain nothing secret, but the run logs and issue history are yours.
- **The .pem lives only in the GitHub secret** (and, if you like, a password manager). Not in Downloads forever, not in the repo, not in a chat.
- **Revoke when done.** Once mnsh1 exists and the workflow is disabled, delete the API key under **My profile > API keys**. GitHub then cannot touch your account at all. Create a new key if you ever need the workflow again.
- **Guard ~/.ssh/oci.** If you lose it, the ways back in are the OCI console's serial console or re-creating the instance.
- **Free means free inside the limits:** 4 OCPU, 24 GB RAM, 200 GB block storage and 10 TB outbound traffic per month, all Always Free. mnsh1 uses 2 / 12 / 200. Don't attach extra block volumes: the storage allowance is already fully used, so a second A1 VM's boot disk would be billable.
- **Idle reclaim.** On plain Free Tier accounts Oracle may reclaim an Always Free VM that sits idle for a week (roughly under 20% CPU, memory and network the whole time). Run something on it, or upgrade the account to *Pay As You Go*, which stays at \$0 while you remain within the free limits and exempts you from reclaim.

Worksheet

Fill this in as you go if you prefer paper. These are identifiers, not secrets. **Never write down the private key or the .pem contents.**

Item	Value
Cloud Account Name (tenancy name)	
Username (email)	
Region identifier (e.g. ap-mumbai-1)	
User OCID, last 8 characters	
Tenancy OCID, last 8 characters	
API key fingerprint	
Public subnet OCID, last 8 characters	
GitHub repo URL	
Public IP of mnsh1	
Workflow disabled? API key deleted?	

Guide version: September 2026. Console labels move around occasionally; if a menu item is not exactly where this says, use the console search box at the top.